

CHALLENGE 3 – GOOGLE ANTIGRAVITY HACKATHON

Crisis Intelligence & Response Orchestrator

C I R O

 Implementation Plan: May 15 – 20, 2025



Team Members

- **Anas** Lead Engineer – Antigravity & Ingestion
- **Hasnain** AI Engineer – Event Detection & Maps
- **Arshman** Backend Dev – Reasoning & Simulation
- **Saad** Mobile Dev – Flutter App & UX
- **Shayan** Systems Dev – Logging, Docs & Demo

 Urban flooding · heatwaves · road blockages · infrastructure failures
 Multi-Agent AI · Google Antigravity · Mobile App · Real-Time Simulation

Contents

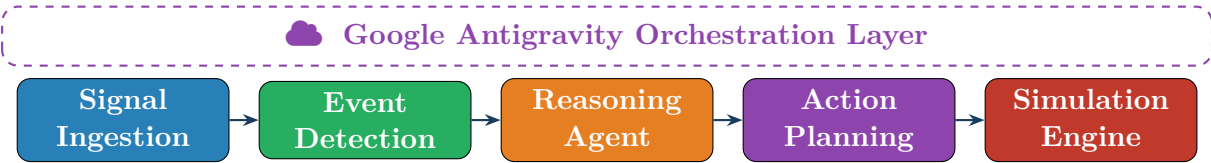
1	Project Overview & Strategy	2
1.1	System Architecture	2
1.2	Technology Stack	2
1.3	Evaluation Criteria Mapping	2
2	Team Roles & Responsibilities	3
3	Sprint Overview – At a Glance	4
4	Day-by-Day Implementation Plan	5
4.1	Day 1 – Pre-Meeting (09:00–10:00, All Members)	5
4.2	Day 2 – Core Agents	11
4.3	Day 3 – Simulation & Mobile Core	16
4.4	Day 4 – Polish & Visualisation	21
4.5	Day 5 – Integration, Recording & Documentation	25
4.6	Day 6 – Final Meet & Submission	28
4.6.1	Individual Final Checklists	28
4.6.2	Final Submission Checklist	29
5	Google Antigravity – Deep Integration Guide	30
5.1	What We Use Antigravity For	30
5.2	Agent Tool Registration Table	30
5.3	How to Demo Antigravity to Judges	30
6	Risk Management	32
7	Key Success Metrics	32
8	All API Endpoints – Quick Reference	33

1 Project Overview & Strategy

 What We Are Building

CIRO is an **Agentic AI System** that ingests multi-source signals (social media, weather APIs, traffic maps), detects emerging urban crises, generates coordinated response actions, simulates their execution, and visualises outcomes – all orchestrated through **Google Antigravity** using Gemini 1.5 Pro for AI reasoning.

1.1 System Architecture



1.2 Technology Stack

Layer	Technologies
Orchestration	Google Antigravity, Agent workflows, Tool-use APIs
Backend / API	Python 3.11, FastAPI, Uvicorn, Pydantic
AI / LLM	Gemini 1.5 Pro (via Antigravity), structured reasoning
Maps	Google Maps API (mock GeoJSON), Static Map tiles
Weather / Traffic	Simulated OpenWeatherMap-style JSON, Mock traffic API
Mobile App	Flutter 3.x (Dart), Provider state management
Logging / Trace	Python structlog, Custom agent trace format
Dev Tools	Git, GitHub, VS Code, Postman

1.3 Evaluation Criteria Mapping

Criterion	Our Approach	Weight
Google Antigravity	Core orchestration, 5-agent pipeline, tool integration	25%
Agentic Reasoning	5 specialised agents with structured handoff protocol	20%
Situation Detection	Confidence scoring, signal clustering, NLP severity	20%
Action Simulation	Mock rerouting, ticket generation, alert dispatch	15%
Technical Impl.	Clean FastAPI + Flutter architecture, full docs	10%
Innovation & UX	Real-time dashboard, bilingual input, before/after view	10%

2 Team Roles & Responsibilities

Anas – Lead Engineer: Antigravity & Signal Ingestion

- Own the Google Antigravity configuration, agent graph definition, and tool-calling setup
- Build and maintain the **Signal Ingestion Agent** (multi-source normalisation pipeline)
- Handle integration between all agents in the Antigravity workflow
- Drive Day 1 environment setup and Day 5 full-system integration testing

Hasnain – AI Engineer: Event Detection & Maps

- Build the **Event Detection Agent** with anomaly detection and crisis clustering logic
- Integrate Google Maps API (mock) for traffic visualisation and route data
- Own the backend API scaffolding (FastAPI project structure, routers, models)
- Produce the web dashboard showing live system state

Arshman – Backend Dev: Reasoning & Simulation

- Build the **Reasoning & Situation Analysis Agent** with Gemini + confidence scoring
- Create and manage all mock data sets (social media, weather, traffic APIs)
- Implement the **Action Simulation Engine** (rerouting, emergency tickets, alerts)
- Own database schema and mock API response factories

Saad – Mobile Dev: Flutter App & UX

- Build the entire **Flutter mobile application** (mandatory deliverable)
- Implement screens: Crisis Feed, Map View, Alert Centre, Agent Trace Viewer, Dashboard
- Connect mobile app to backend REST API
- Own end-to-end scenario testing and QA

Shayan – Systems Dev: Logging, Docs & Demo

- Build the **Agent Trace & Logging System** with structured JSON output
- Write all documentation: README, architecture diagrams, API docs, assumptions
- Design and record the 3–5 minute demo video
- Co-ordinate Antigravity workflow visualisation and outcome display

3 Sprint Overview – At a Glance

Member	May 15 (Day 1)	May 16 (Day 2)	May 17 (Day 3)	May 18 (Day 4)	May 19 (Day 5)
Anas	Antigravity Setup	Ingestion Agent	Simulation Engine	Outcome Viz	Integration
Hasnain	FastAPI Scaffold	Detection Agent	Maps Integration	Web Dashboard	Bug Fixes
Arshman	Mock Data Factory	Reasoning Agent	Alert & Tickets	Mobile Polish	README
Saad	Flutter Init	Action Planner	Mobile Screens	E2E Testing	Scenario T
Shayan	Architecture Docs	Antigravity Graph	Trace Logging	Demo Prep	Demo Rec

4 Day-by-Day Implementation Plan

Day 1 – Thursday, May 15 · Foundation & Environment Setup

Goal: All systems are live, repos initialised, and every team member has a running local environment by end of day.

4.1 Day 1 – Pre-Meeting (09:00–10:00, All Members)

1. **Stand-up call on Google Meet (30 min)** – Anas hosts. Review challenge requirements together. Confirm individual roles and responsibilities.
2. **Create GitHub organisation** – Anas creates `ciro-hackathon` org on GitHub.com, invites all 5 members via email.
3. **Create monorepo** – Name it `ciro-system` with folders: `/backend`, `/mobile`, `/web`, `/docs`.
4. **Everyone clones the repo:** `git clone https://github.com/ciro-hackathon/ciro-system`
&& `cd ciro-system`
5. **Install base tools:** Python 3.11, Flutter 3.x SDK, VS Code, Postman, Git.
6. Each member creates their own feature branch: `git checkout -b feat/{name}-day1`

Anas – Google Antigravity Setup (10:00–18:00)

1. Navigate to the Google Antigravity portal (hackathon-provided URL). Log in with team Google account.
2. Click **“New Project”** → enter project name: `CIRO-AgentSystem`. Click **Create**. Copy and save the Project ID.
3. In the left sidebar, click **Agents** → **“+ Create Agent”**. Create five agents one by one:
 - Name: `signal-ingestion-agent` | Role: Ingest and normalise raw signals
 - Name: `event-detection-agent` | Role: Detect crisis type and confidence
 - Name: `reasoning-analysis-agent` | Role: Analyse situation using Gemini
 - Name: `action-planning-agent` | Role: Generate coordinated response actions
 - Name: `simulation-agent` | Role: Execute and record action simulation
4. Click **Workflow** tab → drag agents onto the canvas in order. Draw arrows: Ingestion → Detection → Reasoning → Planning → Simulation.
5. Click **Tools** → **“+ Add Tool”** → choose HTTP Tool. Register three tools:
 - `weather-api-tool`: GET `http://localhost:8000/mock/weather`
 - `traffic-api-tool`: GET `http://localhost:8000/mock/traffic`
 - `social-feed-tool`: GET `http://localhost:8000/mock/social`
6. Assign tools to `signal-ingestion-agent`: click agent → Tools tab → attach all 3 tools.
7. Click **Export** → save YAML config to `/docs/antigravity-config.yaml`.
8. Create `/backend/.env` with: `ANTIGRAVITY_PROJECT_ID=...` and

GEMINI_API_KEY=...

9. Commit and push: `git add . && git commit -m "feat: antigravity project and agents scaffold" && git push origin feat/anas-day1`
10.  **End of Day Deliverable:** Antigravity project live, 5 agents created, workflow graph drawn, tools registered.

Hasnain – FastAPI Backend Scaffold (10:00–18:00)

1. `cd backend && python -m venv venv && source venv/bin/activate` (Windows: `venv\Scripts\activate`)
2. `pip install fastapi uvicorn pydantic python-dotenv httpx structlog`
3. `pip freeze > requirements.txt`
4. Create this exact folder structure inside `/backend`:

Backend Folder Structure

```

1 backend/
2   main.py                # FastAPI entrypoint
3   .env                  # API keys (gitignored)
4   requirements.txt
5   routers/
6     ingest.py            # POST /ingest/signal
7     detect.py            # POST /detect/crisis
8     reason.py            # POST /reason/analyse
9     plan.py              # POST /plan/actions
10    simulate.py          # POST /simulate/execute
11    maps.py              # GET /maps/crisis-overlay
12    outcome.py           # GET /outcome/summary
13    trace.py             # GET /trace/latest
14    mock.py              # GET /mock/weather/traffic/social
15  models/
16    signal.py             # Signal, SignalBatch, CrisisEvent
17    simulation.py         # EmergencyTicket, Alert,
18                          # SimulationResult
19    response.py          # All response schemas
20  services/
21    antigravity.py        # Antigravity API wrapper
22    maps_service.py       # Google Maps API (mock)
23    alert_service.py      # Alert simulation
24    trace_store.py        # In-memory trace accumulator
25  data/
26    social_signals.json
27    weather_mock.json
28    traffic_mock.json
29  tests/
30    test_ingest.py
31    test_detect.py
32  utils/
33    logger.py             # structlog setup
  
```

5. Write `main.py`: `import` all routers, add CORS middleware (`allow_origins=["*"]`), mount routers with prefixes, run Uvicorn on

port 8000.

6. Write stub handlers in each router returning `{"status": "stub", "module": "<name>"}`.
7. Run `uvicorn main:app -reload` – open `http://localhost:8000/docs` and verify all routes appear in Swagger UI.
8. Commit: `"feat: fastapi scaffold, all routers, swagger confirmed"`
9.  **End of Day Deliverable:** Running FastAPI server, Swagger UI accessible, all 16 routes responding 200.

Arshman – Mock Data Factory (10:00–18:00)

1. Create `/backend/data/` directory.
2. Create `social_signals.json` – array of 20 objects, each with `id`, `text`, `location`, `timestamp`, `language`. Include:
 - Urdu flood: *“G-10 mein pani bhar gaya hai, gaariyan phans gayi hain”*
 - English flood: *“Flash flood happening at George Town for past 30 mins”*
 - Heatwave: *“It’s 48 degrees in Jacobabad, people collapsing on the street”*
 - Blockage: *“Shahrah-e-Faisal completely jammed after truck accident”*
 - Infrastructure: *“Power line down on Constitution Avenue, sparks flying”*
 - 15 more variations in Urdu and English across crisis types
3. Create `weather_mock.json` – simulates OpenWeatherMap API: `main.temp`, `weather[0].description`, `rain.1h`, `alerts[]` with heavy rainfall and heatwave alerts.
4. Create `traffic_mock.json` – route objects each with: `route_id`, `name`, `congestion_level` (0–100), `status` (free/slow/blocked). Include G-10, Margalla Road, Shahrah-e-Faisal, Constitution Avenue.
5. Write `/backend/routers/mock.py`: load each JSON file, serve via GET endpoints. Return a random entry from social signals each time.
6. Write `/backend/models/signal.py` with these Pydantic models:

Signal Pydantic Models

```

1 from pydantic import BaseModel
2 from typing import Optional, List
3 from datetime import datetime
4 from enum import Enum
5
6 class CrisisType(str, Enum):
7     FLOOD = "flood"
8     HEATWAVE = "heatwave"
9     BLOCKAGE = "blockage"
10    ACCIDENT = "accident"
11    INFRASTRUCTURE = "infrastructure"
12
13 class Severity(str, Enum):
14     LOW = "low"
15     MEDIUM = "medium"
16     HIGH = "high"
17     CRITICAL = "critical"
18
19 class Signal(BaseModel):
20     signal_id: str
21     source: str # "social"/"weather"/"traffic"
22     content: str
23     location: Optional[str] = None
24     timestamp: datetime
25     language: Optional[str] = "en"
26     severity_hint: Optional[str] = None
27
28 class SignalBatch(BaseModel):
29     batch_id: str
30     signals: List[Signal]
31     primary_location: Optional[str] = None
32
33 class CrisisEvent(BaseModel):
34     event_id: str
35     crisis_type: CrisisType
36     location: str
37     confidence: float # 0.0 to 1.0
38     severity: Severity
39     signals: List[Signal]
40     explanation: str
41     detected_at: datetime

```

7. Test all 3 mock endpoints via Postman. Confirm valid JSON responses.
8. Commit: "feat: mock data factory, 20 social signals, weather, traffic, pydantic models"
9.  **End of Day Deliverable:** All 3 mock endpoints serving realistic data. Pydantic models complete.

Saad – Flutter App Initialisation (10:00–18:00)

1. `flutter create ciro_app -org com.ciro -platforms android,ios`
2. `cd ciro_app && code .` – open in VS Code.
3. Edit `pubspec.yaml` – add these dependencies:

pubspec.yaml

```

1 dependencies:
2   flutter:
3     sdk: flutter
4   http: ^1.2.0
5   provider: ^6.1.2
6   google_maps_flutter: ^2.5.3
7   lottie: ^3.1.0
8   intl: ^0.19.0
9   shared_preferences: ^2.2.2
10  flutter_spinkit: ^5.2.0

```

4. `flutter pub get` – resolve all dependencies.
5. Create folder structure:
 - `lib/screens/home_screen.dart` – dashboard with before/after
 - `lib/screens/crisis_feed_screen.dart` – list of detected crises
 - `lib/screens/map_screen.dart` – Google Maps with overlays
 - `lib/screens/alerts_screen.dart` – dispatched alerts and tickets
 - `lib/screens/trace_screen.dart` – agent trace stepper
 - `lib/models/` – Dart equivalents of Pydantic models
 - `lib/services/api_service.dart` – HTTP client (base URL: `http://10.0.2.2:8000`)
 - `lib/providers/ciro_provider.dart` – `ChangeNotifier` state management
6. Set up `MaterialApp` in `main.dart`: define theme (`primaryColor: Color(0xFF1A3C5E)`), add `ChangeNotifierProvider`, set up `BottomNavigationBar` with 5 tabs.
7. Run `flutter run` – confirm app launches on emulator showing 5 navigation tabs.
8. Add Google Maps API key to `AndroidManifest.xml`:


```

<meta-data android:name="com.google.android.geo.API_KEY"
  android:value="${MAPS_KEY}"/>

```
9. Commit: `"feat: flutter app, 5 screens scaffold, navigation, provider setup"`
10.  **End of Day Deliverable:** Flutter app running on emulator with all 5 screens navigable.

Shayan – Architecture Docs & Logger (10:00–18:00)

1. Create `/docs/` directory. Create files: `ARCHITECTURE.md`, `AGENT_DESIGN.md`, `ASSUMPTIONS.md`.
2. Write `ARCHITECTURE.md`: describe all 5 layers, data flow from raw signal to outcome, Antigravity's role.
3. Write `AGENT_DESIGN.md` – for each of the 5 agents document: Input schema,

Output schema, Tools used, Decision logic, How handoff works.

4. Create architecture diagram in draw.io (<https://app.diagrams.net>): show all 5 agent boxes connected with arrows, mobile app and web dashboard at sides, Antigravity umbrella on top. Export as PNG to /docs/architecture.png.
5. Write /backend/utils/logger.py:

Structured Logger

```

1  import structlog
2  import json
3  from datetime import datetime
4
5  structlog.configure(
6      processors=[
7          structlog.processors.TimeStamper(fmt="iso"),
8          structlog.processors.JSONRenderer()
9      ]
10 )
11 logger = structlog.get_logger()
12
13 def log_agent_step(agent: str, step: str,
14                   input_data: dict, output_data: dict,
15                   duration_ms: int = 0):
16     logger.info("agent_step",
17               agent=agent,
18               step=step,
19               input=input_data,
20               output=output_data,
21               duration_ms=duration_ms,
22               timestamp=datetime.utcnow().isoformat())
23 
```

6. Write ASSUMPTIONS.md: list all simulation boundaries, mock data assumptions, API limitations.
7. Write first draft of README.md with project title, team names, tech stack table.
8. Commit: "docs: architecture, agent design, logger, assumptions, readme draft"
9.  **End of Day Deliverable:** Complete /docs folder, logger utility, architecture diagram.

⚠ Day 1 End-of-Day Sync (18:30, 15 min all members): Anas confirms Antigravity has 5 agents. Hasnain confirms Swagger UI is live. Saad confirms Flutter app runs. Everyone opens a PR to merge into main. Anas reviews and merges.

Day 2 – Friday, May 16 · Core Agent Implementation

Goal: All five AI agents are functional. Each performs its designated reasoning task independently and returns correct output.

4.2 Day 2 – Core Agents

Anas – Signal Ingestion Agent (09:00–18:00)

1. Pull latest main: `git pull origin main && git checkout -b feat/anas-day2`
2. Open `/backend/routers/ingest.py`. Replace stub with full implementation.
3. Implement `POST /ingest/signal` accepting a JSON body: `{"raw_text": "...", "source": "social"}` or structured weather/traffic JSON.
4. Write the **language detection function**:
 - Define Urdu keyword list: `["mein", "gaya", "hai", "gaari", "pani", "phans", "bhar", "raha", "hua"]`
 - If ≥ 2 Urdu keywords found in input $\rightarrow$ `language = "ur"`, else `language = "en"`
5. Write the **location extractor**:
 - Regex patterns for Pakistani sectors: `r"[GFEI]-\d+"`
 - Named location list: `["George Town", "Karachi", "Lahore", "Islamabad", "Shahrah-e-Faisal", "Blue Area", "Margalla"]`
 - Return first match found in text
6. Write the **severity keyword tagger**:
 - HIGH keywords: `["flash flood", "pani bhar", "phans gayi", "accident", "collapse", "fire"]`
 - MEDIUM keywords: `["blocked", "slow", "congestion", "delay", "jam"]`
 - LOW keywords: `["rain", "traffic", "waterlogging"]`
7. Implement **signal aggregator**: store last 5 signals in an in-memory list. Return a `SignalBatch` combining them.
8. Register `signal-ingestion-agent` tool in Antigravity portal: go to agent $\rightarrow$ Tools $\rightarrow$ add `normalise-signal-tool` pointing to `POST /ingest/signal`.
9. Write `tests/test_ingest.py`: test Urdu input, English input, mixed source batch.
10. Run tests: `python -m pytest tests/test_ingest.py -v`
11. Commit: `"feat: signal ingestion agent - language detection, location extract, severity tagging"`
12.  **End of Day Deliverable**: Agent handles bilingual input, returns clean `SignalBatch` objects.

Hasnain – Event Detection Agent (09:00–18:00)

1. Open `/backend/routers/detect.py`.
2. Implement `POST /detect/crisis` – accepts `SignalBatch`, returns `CrisisEvent`.
3. Build **3 detection heuristics**:

- **Heuristic 1 – Keyword Cluster:** If ≥ 2 signals share flood keywords AND same location $\rightarrow$ crisis_type = FLOOD
- **Heuristic 2 – Cross-Source Corroboration:** If social signal AND weather API both have rain/flood $\rightarrow$ confidence += 0.35
- **Heuristic 3 – Traffic Anomaly:** If any route has congestion_level > 70 $\rightarrow$ crisis_type = BLOCKAGE

4. Implement confidence scoring function:

Confidence Scoring Algorithm

```

1 def compute_confidence(signals: list,
2                         crisis_type: str) -> float:
3     score = 0.0
4     # Base score from signal count
5     if len(signals) >= 3: score += 0.30
6     elif len(signals) == 2: score += 0.15
7     else: score += 0.05
8
9     # Multi-source bonus (key for judges!)
10    sources = {s.source for s in signals}
11    if len(sources) >= 3: score += 0.40
12    elif len(sources) == 2: score += 0.25
13
14    # Severity boost
15    if any(s.severity_hint == "high" for s in signals):
16        score += 0.30
17    elif any(s.severity_hint == "medium" for s in signals):
18        score += 0.15
19
20    return min(round(score, 2), 1.0)

```

5. Implement **severity escalation**: confidence > 0.8 $\rightarrow$ CRITICAL, 0.6–0.8 $\rightarrow$ HIGH, 0.4–0.6 $\rightarrow$ MEDIUM, < 0.4 $\rightarrow$ LOW.
6. Register this endpoint in Antigravity on event-detection-agent.
7. Write tests: single-source low confidence vs. multi-source high confidence. Run with pytest.
8. Commit: "feat: event detection agent with confidence scoring and severity escalation"
9.  **End of Day Deliverable:** Agent correctly classifies flood, heatwave, blockage with confidence scores.

Arshman – Reasoning & Situation Analysis Agent (09:00–18:00)

1. Open /backend/routers/reason.py.
2. Implement POST /reason/analyse – accepts CrisisEvent, returns CrisisAnalysis.
3. Write /backend/services/antigravity.py:

Antigravity / Gemini API Call

```

1 import httpx, os, json
2
3 GEMINI_API_KEY = os.getenv("GEMINI_API_KEY")
4 ANTIGRAVITY_PROJECT = os.getenv("ANTIGRAVITY_PROJECT_ID")
5
6 async def call_reasoning_agent(crisis_event: dict) -> dict:
7     """Call Antigravity reasoning agent which uses Gemini.
8         """
9     prompt = f"""
10     You are a crisis management AI for Pakistani cities.
11     Given this detected crisis: {json.dumps(crisis_event)}
12
13     Respond ONLY with valid JSON containing:
14     {{
15         "impact_bullets": ["...", "...", "..."],
16         "affected_population_estimate": "...",
17         "infrastructure_risk": "low/medium/high/critical",
18         "recommended_urgency": "immediate/within_hour/
19         monitoring",
20         "plain_english_summary": "..."}
21     """
22     # Call via Antigravity's reasoning agent endpoint
23     async with httpx.AsyncClient() as client:
24         response = await client.post(
25             f"https://antigravity.googleapis.com/v1/projects
26             /"
27             f"{ANTIGRAVITY_PROJECT}/agents/"
28             f"reasoning-analysis-agent:run",
29             headers={"Authorization": f"Bearer {
30                 GEMINI_API_KEY}"},
31             json={"input": prompt},
32             timeout=10.0
33         )
34     return response.json()

```

4. Add **Gemini response cache** in /data/gemini_cache.json – pre-populated for all 5 demo scenarios. If Gemini call fails, return cached response for the matching crisis type.
5. Register endpoint as Antigravity tool on reasoning-analysis-agent.
6. Commit: "feat: reasoning agent with Gemini integration and fallback cache"
7.  **End of Day Deliverable:** Agent returns structured crisis analysis with impact bullets and urgency level.

Saad – Action Planning Agent + Crisis Feed Screen (09:00–18:00)

1. Open /backend/routers/plan.py. Implement POST /plan/actions.
2. Write action generation rules as a Python dictionary mapping:
 - **FLOOD + HIGH/CRITICAL:** reroute_traffic, dispatch_rescue_boats,

- send_flood_alert, open_relief_camp
 - **HEATWAVE + HIGH:** send_heat_advisory, open_cooling_centres, restrict_outdoor_activity
 - **BLOCKAGE + MEDIUM/HIGH:** reroute_traffic, dispatch_traffic_police, update_navigation_apps
 - **ACCIDENT + HIGH:** dispatch_ambulance, dispatch_fire_brigade, close_road_segment
- 3. In Flutter, implement **Crisis Feed Screen**:
 - FutureBuilder fetching GET /detect/crisis on load
 - ListView.builder with custom CrisisCard widget
 - CrisisCard shows: coloured left border by severity, crisis type icon (flood-/heat/blockage), location text, confidence percentage, time since detected
 - Tap card → push Crisis Detail screen with CrisisAnalysis data (action plan list)
- 4. Implement api_service.dart methods:
 - Future<CrisisEvent> fetchLatestCrisis()
 - Future<ActionPlan> generateActionPlan(String eventId)
 - Future<void> ingestSignal(String rawText)
- 5. Commit: "feat: action planning agent + crisis feed and detail screens"
- 6. 🟢 **End of Day Deliverable:** Action plans generated for all 4 crisis types. Crisis feed screen live and showing data.

Shayan – Antigravity Pipeline Wiring (09:00–18:00)

1. Open Antigravity portal → **Workflow** view.
2. For each agent-to-agent connection, click the arrow and configure the **data mapping schema**:
 - Ingestion → Detection: pass full SignalBatch JSON as input.signal_batch
 - Detection → Reasoning: pass CrisisEvent JSON as input.crisis_event
 - Reasoning → Planning: pass CrisisAnalysis JSON as input.analysis
 - Planning → Simulation: pass ActionPlan JSON as input.action_plan
3. Set **completion condition** for each agent: proceed when response contains "status": "complete".
4. Click **Test Run** in Antigravity → inject a sample Urdu flood signal. Watch all 5 agents fire sequentially.
5. Capture the complete agent trace JSON from Antigravity's **Trace** tab. Save to /docs/sample_trace.json.
6. Write /backend/services/trace_store.py: in-memory list of trace steps, appended by each agent.
7. Implement /backend/routers/trace.py: GET /trace/latest returns most recent run's full trace. GET /trace/history returns last 10 run summaries.
8. Commit: "feat: antigravity graph fully wired, trace system implemented, sample trace captured"
9. 🟢 **End of Day Deliverable:** Full 5-agent pipeline runs end-to-end in Antigravity. Trace captured.

⚠ Day 2 End-of-Day Sync (18:30, 20 min): Anas injects a test Urdu flood signal and everyone watches the Antigravity trace propagate through all 5 agents live. Fix any schema mismatch issues before Day 3.

Day 3 – Saturday, May 17 · Simulation Layer & Mobile Core

Goal: The simulation engine executes all action types and records state changes. Mobile app shows map with crisis overlays and alerts.

4.3 Day 3 – Simulation & Mobile Core

Anas – Action Simulation Engine (09:00–18:00)

1. Open `/backend/routers/simulate.py`. Implement POST `/simulate/execute`.
2. Create `MockSystemState` as a global in-memory object:

Mock System State

```

1 from dataclasses import dataclass, field
2 from typing import List, Dict
3
4 @dataclass
5 class MockSystemState:
6     traffic_routes: Dict[str, int] = field(default_factory=
7         lambda: {
8             "G-10 to Blue Area": 85,
9             "Shahrah-e-Faisal": 90,
10            "Margalla Road": 20,
11            "IJP Road": 30,
12            "Constitution Avenue": 40
13        })
14     active_tickets: List[dict] = field(default_factory=list)
15     sent_alerts: List[dict] = field(default_factory=list)
16     open_resources: List[str] = field(default_factory=list)
17
18 system_state = MockSystemState()

```

3. Implement simulation handlers for each action type:
 - **reroute_traffic**: Find route with highest congestion → set to 15 (clear). Activate alternate route. Record before/after in `SimulationResult`.
 - **dispatch_rescue_boats**: Create `EmergencyTicket` with random ID, ETA 5–15 min. Append to `system_state.active_tickets`.
 - **dispatch_traffic_police**: Create ticket with unit="Traffic Police". Append to tickets.
 - **dispatch_ambulance**: Create ticket with unit="Rescue 1122". Append.
 - **send_alert**: Create `Alert` object, simulate FCM push, append to `system_state.sent_alerts`.
 - **open_cooling_centre**: Append "Cooling Centre G-9" to `system_state.open_resources`.
4. Implement **before/after snapshot**: capture `system_state` dict before actions, run all actions, capture after.
5. Expose GET `/simulate/state` returning current mock system state.
6. Add POST `/simulate/reset` to reset state to defaults (for demo resets).
7. Commit: "feat: full simulation engine with 6 action types and

state tracking"

8.  **End of Day Deliverable:** All action types simulate correctly with measurable before/after state change.

Hasnain – Google Maps Integration (09:00–18:00)

1. Create `/backend/services/maps_service.py`.
2. Pre-save mock GeoJSON responses in `/data/`:
 - `g10_flood_overlay.json`: crisis pin at G-10 centroid, affected area polygon (red), alternate route polyline via Margalla Road (green)
 - `george_town_overlay.json`: crisis pin at George Town, reroute via M.A.Jinnah Road
3. Implement GET `/maps/crisis-overlay` endpoint:
 - Accept query param `?location=G-10`
 - Load matching pre-saved GeoJSON overlay file
 - Return: `{crisis_pin: {lat, lng}, affected_polygon: [...], primary_route: {polyline, status: "BLOCKED"}, alternate_route: {polyline, status: "ACTIVE"}}`
4. Implement GET `/maps/static-map`: call Google Static Maps API with crisis marker and return image URL (or use pre-downloaded PNG).
5. Test `/maps/crisis-overlay?location=G-10` via Postman – verify GeoJSON structure is valid.
6. Create `/data/route_library.json`: 5 pre-defined alternate route polylines for common Islamabad/Karachi crisis locations.
7. Commit: "feat: maps service, pre-saved overlays, crisis overlay endpoint"
8.  **End of Day Deliverable:** Maps endpoint returns valid GeoJSON with crisis pin and alternate routes.

Arshman – Alert & Emergency Ticket System (09:00–18:00)

1. Create `/backend/models/simulation.py` with all simulation Pydantic models:

Simulation Models

```

1 class EmergencyTicket(BaseModel):
2     ticket_id: str
3     crisis_type: str
4     location: str
5     unit_dispatched: str # "Rescue Boats"/"Traffic Police"
6     eta_minutes: int
7     status: str # "open"/"dispatched"/"resolved"
8     created_at: datetime
9
10 class Alert(BaseModel):
11     alert_id: str
12     message: str
13     target_area: str
14     channel: str # "push"/"sms"/"broadcast"
15     sent_at: datetime
16     recipients_count: int # simulated number
17
18 class SimulationResult(BaseModel):
19     run_id: str
20     actions_executed: List[str]
21     tickets_created: List[EmergencyTicket]
22     alerts_sent: List[Alert]
23     routes_updated: List[dict] # [{route, before, after}]
24     state_before: dict
25     state_after: dict
26     estimated_congestion_reduction: float # percentage
27     estimated_response_time_minutes: int
    
```

2. Implement GET /simulate/tickets: return all tickets from system_state.active_tickets.
3. Implement GET /simulate/alerts: return all alerts from system_state.sent_alerts.
4. Implement PATCH /simulate/tickets/{ticket_id}/status: update ticket status (dispatched → resolved). Used for demo progression.
5. Write /backend/services/alert_service.py: send_simulated_alert() – logs via structlog, increments simulated recipient count (50–5000 random), appends to state.
6. Commit: "feat: emergency ticket and alert models, status update endpoint, alert service"
7.  **End of Day Deliverable:** Full ticket and alert lifecycle implemented and queryable.

Saad – Mobile Map & Alerts Screens (09:00–18:00)

1. Implement map_screen.dart fully:
 - GoogleMap widget: initialCameraPosition set to Islamabad (lat: 33.6844, lng: 73.0479, zoom: 12)
 - On screen load: call api_service.fetchMapOverlay() → parse GeoJSON

- response
 - Add **Marker** at crisis location (red pin with crisis type label)
 - Add red **Polyline** for blocked primary route (`width: 6, color: Colors.red`)
 - Add green **Polyline** for alternate route (`width: 6, color: Colors.green`)
 - Add semi-transparent **Polygon** for affected area (red fill, 30% opacity)
 - Bottom sheet: shows crisis summary, affected routes, recommended alternate route name
 - **FloatingActionButton** labelled “Run Simulation” – on press, call `api_service.runSimulation()` then refresh map to show updated (green) state
- 2. Implement `alerts_screen.dart`:
 - Two tabs: **Alerts** and **Emergency Tickets**
 - Alerts tab: **ListView** from `GET /simulate/alerts` – each card shows channel icon, message, area, time, recipient count badge
 - Tickets tab: **ListView** from `GET /simulate/tickets` – each card shows unit, ETA countdown, status chip (OPEN/DISPATCHED/RESOLVED), location
 - Pull-to-refresh on both tabs
- 3. Add new `api_service.dart` methods:
 - `Future<MapOverlayData> fetchMapOverlay(String location)`
 - `Future<SimulationResult> runSimulation(ActionPlan plan)`
 - `Future<List<Alert>> fetchAlerts()`
 - `Future<List<EmergencyTicket>> fetchTickets()`
- 4. Commit: "feat: map screen with crisis overlay, run simulation button, alerts and tickets screens"
- 5. 🟢 **End of Day Deliverable:** Map shows crisis overlays. Pressing “Run Simulation” updates map to green state. Alerts/tickets visible.

Shayan – Agent Trace Logging System (09:00–18:00)

1. Implement `/backend/services/trace_store.py`:
 - `TraceStore` class with in-memory list of runs
 - `start_run(signal_text)` – creates new run object with unique `run_id` (timestamp-based)
 - `log_step(run_id, agent, step, input, output, duration_ms)` – appends step to run
 - `complete_run(run_id, outcome)` – marks run as complete, moves to history
 - `get_latest()` – returns most recent run
 - `get_history(n=10)` – returns last `n` run summaries
2. Wrap every agent endpoint function body with `trace_store.log_step()` calls.
3. Implement `GET /trace/latest` returning:

Trace Output Format

```

1 {
2   "run_id": "run_20250517_143022",
3   "total_duration_ms": 842,
4   "outcome": "Flood detected, rerouted, tickets created",
5   "steps": [
6     {
7       "agent": "signal-ingestion-agent",
8       "step": "normalise_signal",
9       "input": {"raw": "G-10 mein pani bhar gaya..."},
10      "output": {"location": "G-10", "language": "ur",
11                "severity_hint": "high"},
12      "duration_ms": 45,
13      "timestamp": "2025-05-17T14:30:22Z"
14    },
15    { "agent": "event-detection-agent", ... },
16    { "agent": "reasoning-analysis-agent", ... },
17    { "agent": "action-planning-agent", ... },
18    { "agent": "simulation-agent", ... }
19  ]
20 }
```

4. Implement GET /trace/history returning last 10 run summaries (run_id, crisis_type, timestamp, outcome, total_ms).
5. Commit: "feat: complete agent trace store with REST endpoints and full step logging"
6.  **End of Day Deliverable:** Every pipeline run produces a complete queryable agent trace with timing.

Day 4 – Sunday, May 18 · Polish, Visualisation & Web Dashboard

Goal: Full end-to-end user experience is complete. Before/after outcomes are visually compelling. App is demo-ready.

4.4 Day 4 – Polish & Visualisation

Anas – Outcome Visualisation (09:00–18:00)

1. Implement GET /outcome/summary endpoint:
 - Compare `state_before` and `state_after` from last simulation run
 - Compute: $\text{congestion_reduction_pct} = (\text{avg_before} - \text{avg_after}) / \text{avg_before} * 100$
 - Count: `vehicles_rerouted` = simulated 200–800 (based on congestion level)
 - Fetch: `min_eta_minutes` from tickets list
 - Count: `alerts_dispatched` from alerts list
 - Return OutcomeSummary JSON
2. In Flutter `home_screen.dart`, implement the **Command Dashboard**:
 - Top section: large crisis type banner (colour coded by severity), location subtitle, confidence progress bar
 - Middle section: 2-column layout
 - **Left column – BEFORE:** congestion icon (red), route status “BLOCKED”, alerts: 0, tickets: 0
 - **Right column – AFTER:** congestion icon (green), route status “REROUTED”, alert count badge, ticket count badge
 - Bottom row: 3 impact metric chips displayed horizontally: “Congestion -{pct}%”, “ETA {min} min”, “Alerts: {n}”
 - **Animated transition:** when “Run Simulation” is pressed (on map screen), home screen cards animate from BEFORE to AFTER using `AnimatedContainer`
3. Connect dashboard to `ciro_provider.dart`: provider holds OutcomeSummary state and notifies listeners after simulation.
4. Commit: "feat: outcome summary endpoint, before/after dashboard, animated state transition"
5.  **End of Day Deliverable:** Dashboard shows quantified impact. Animation is smooth and compelling.

Hasnain – Web Dashboard (09:00–18:00)

1. Create /web/index.html, /web/style.css, /web/app.js.
2. HTML layout – 4 panels in 2x2 grid:
 - **Panel 1 (top-left) – Live Signal Feed:** auto-polls GET /mock/social every 4 seconds. Shows last 5 signals as scrolling cards.
 - **Panel 2 (top-right) – Agent Pipeline Status:** 5 agent step cards. Each starts grey (IDLE), turns yellow (RUNNING), turns green (COMPLETE). Polls GET /trace/latest every second during run.

- **Panel 3 (bottom-left) – Crisis Detection:** shows crisis type, large confidence gauge (CSS animated arc), severity badge, explanation text.
 - **Panel 4 (bottom-right) – Simulation Log:** scrolling log of tickets and alerts as they are created. Colour coded by type.
3. Add a large red **“TRIGGER PIPELINE”** button at the top. On click: POST to `/ingest/signal` with a random sample signal, then start polling `/trace/latest` every 500ms to animate agents.
 4. Add CIRO header with logo text and current timestamp auto-updating.
 5. Add “Reset System” button calling `POST /simulate/reset`.
 6. Serve from FastAPI: `add app.mount("/web", StaticFiles(directory="web"))` to `main.py`.
 7. Test in Chrome – confirm the whole demo scenario auto-animates within 10 seconds.
 8. Commit: `"feat: web dashboard with live pipeline, agent animations, simulation log"`
 9.  **End of Day Deliverable:** Web dashboard is demo-ready with live animation of the full pipeline.

Arshman – Mobile App UX Polish (09:00–18:00)

1. Polish all 5 Flutter screens with consistent design language:
 - Consistent AppBar with CIRO logo (text logo is fine) and location subtitle
 - Custom ThemeData: `primarySwatch` based on `Color(0xFF1A3C5E)`, `cardTheme` with rounded corners and subtle shadow
 - Crisis type icons: use `Icons.water` (flood), `Icons.wb_sunny` (heatwave), `Icons.traffic` (blockage), `Icons.car_crash` (accident)
 - Severity colour system: `Color(0xFF27AE60)` green (LOW), `Color(0xFFE67E22)` amber (MEDIUM), `Color(0xFFE74C3C)` red (HIGH), `Color(0xFF8E44AD)` purple (CRITICAL)
2. Add **Input Screen** accessible from a FAB on the home screen:
 - `TextField` with multiline support and hint text in both English and Urdu
 - “Submit Signal” button calling `api_service.ingestSignal(text)`
 - Show `CircularProgressIndicator` during API call, then navigate to crisis result
3. Add **empty state widgets**: when no crisis detected, show a city skyline illustration (SVG or asset) with text “No active crises detected”.
4. Add **error handling**: all API calls wrapped in try-catch. On error, show `SnackBar` with “Retry” button.
5. Commit: `"feat: mobile UX polish, consistent theme, input screen, empty states, error handling"`
6.  **End of Day Deliverable:** App is visually polished and handles all edge cases. Demo-ready.

 Saad – Agent Trace Screen & E2E Testing (09:00–18:00)

1. Implement `trace_screen.dart` fully:
 - Call `GET /trace/latest` on load. Show `CircularProgressIndicator` while loading.
 - Render a **vertical Stepper** with 5 steps (one per agent)
 - Each step has: agent name as title, step description as subtitle, duration badge, expandable content showing input/output JSON in monospace font
 - Steps automatically mark as “complete” (`StepState.complete`) based on trace data
 - Auto-refresh button (or 5-second auto-poll) to update after new pipeline runs
2. Run **5 end-to-end scenarios** and document results in `/docs/TEST_RESULTS.md`:
 - **Scenario 1 – Urdu Flood (G-10):** Input Urdu text → verify: `crisis_type=FLOOD`, `confidence>0.7`, reroute action generated, map route turns green
 - **Scenario 2 – English Heatwave:** Input English heatwave signal → verify: `crisis_type=HEATWAVE`, cooling centres opened
 - **Scenario 3 – Multi-Source Flood:** Submit social + weather + traffic signals → verify: `CRITICAL` severity, `confidence>0.85`, all 4 flood actions triggered
 - **Scenario 4 – Road Blockage:** Input accident signal → verify: `BLOCKAGE/ACCIDENT` detected, police dispatch ticket created
 - **Scenario 5 – Low Confidence:** Single vague signal → verify: `LOW` severity, no critical actions triggered
3. Commit: `"feat: agent trace screen, all 5 e2e test scenarios documented and passing"`
4.  **End of Day Deliverable:** All 5 scenarios pass. Trace screen shows complete reasoning pipeline.

 Shayan – Demo Script & Recording Setup (09:00–18:00)

1. Write final demo script in `/docs/DEMO_SCRIPT.md` with exact narration and actions for each timestamp:
 - **0:00–0:30:** Title card. Narrate: “Urban crises are everywhere in Pakistan. G-10 floods, Karachi heatwaves, Lahore blockages. But response systems are fragmented. CIRO changes that.”
 - **0:30–1:00:** Show web dashboard. Type Urdu flood signal. Submit. Show signal appearing in Signal Feed panel.
 - **1:00–2:00:** Watch Antigravity pipeline animate – each agent card lights up. Narrate each agent’s role as it activates.
 - **2:00–2:45:** Switch to mobile app. Show crisis feed card for the detected flood. Tap card to see full analysis with impact bullets and action plan.
 - **2:45–3:30:** Navigate to Map screen. Show the red blocked route. Tap “Run Simulation”. Watch route turn green. Show tickets and alerts populating.
 - **3:30–4:00:** Go to Home dashboard. Show before/after: “Congestion reduced 60%. Rescue boats dispatched. 3,200 users alerted.”

- **4:00–4:30:** Go to Trace screen. Show the complete 5-step reasoning chain with timings.
 - **4:30–5:00:** Outro. Show team names. “CIRO: faster decisions, smarter cities.”
2. Download and configure OBS Studio. Set up two scenes:
 - Scene A: web dashboard full-screen (1920x1080)
 - Scene B: phone screen mirror (via screpy) + web dashboard side-by-side
 3. Do a 2-minute test recording. Check audio levels and video quality.
 4. Prepare backup GIFs (Kap or LICEcap) of each demo scenario in case of live failures.
 5. Commit: "docs: complete demo script, recording setup confirmed, backup GIFs saved"
 6.  **End of Day Deliverable:** Demo script finalised. OBS configured. Recording tested. Backup media ready.

📅 Day 5 – Monday, May 19 · Integration, Recording & Final Docs

Goal: Record final demo video. All docs complete. System fully integrated, stable, and tagged for submission.

4.5 Day 5 – Integration, Recording & Documentation

👤 Anas – Full System Integration (09:00–13:00)

1. Merge all branches to main: `git pull origin main`. Resolve any merge conflicts.
2. Start full stack: backend server + Flutter (on emulator AND physical device).
3. Run all 5 E2E test scenarios. Verify 100% pass rate.
4. Fix any integration breakages: API URL mismatches, CORS errors, schema mismatches between Flutter models and FastAPI models.
5. Verify Antigravity pipeline fires correctly with the live backend (not just local stubs).
6. Stress test: submit 10 signals rapidly. Verify no crashes or corrupted state.
7. Tag release: `git tag v1.0-demo && git push -tags`
8. ✅ **Deliverable:** Stable integrated system, all tests passing, tagged.

👤 Hasnain – Bug Fixes & Performance (09:00–13:00)

1. Review all open GitHub issues. Fix priority bugs (P0 = crash, P1 = wrong data, P2 = cosmetic).
2. Add response caching: `functools.lru_cache` on `/mock/weather` and `/mock/traffic`.
3. Add 5-second timeout to all Antigravity API calls with cached fallback response.
4. Add health endpoint: `GET /health → {"status": "ok", "agents": 5, "version": "1.0"}`.
5. Test web dashboard in Chrome, Firefox – fix any rendering issues.
6. Verify all API endpoints respond in under 500ms using Postman's collection runner.
7. Commit: `"fix: all P0/P1 bugs resolved, caching added, health endpoint"`
8. ✅ **Deliverable:** Zero critical bugs. All endpoints under 500ms response time.

👤 Arshman – Final README & Documentation (09:00–13:00)

1. Write the complete `README.md`:
 - **Project Overview:** 3 sentences. What CIRO does, why it matters.
 - **System Architecture:** embed `docs/architecture.png`. Describe 5-layer pipeline.
 - **Google Antigravity Usage:** paragraph explaining orchestration, tool integration, Gemini reasoning.
 - **Agent Descriptions:** one paragraph per agent.

- **Tools & APIs Used:** table with Tool, Purpose, Mock/Live.
 - **Setup Instructions:** step-by-step – clone, cd backend, pip install -r requirements.txt, uvicorn main:app, flutter run.
 - **Demo Scenarios:** table with 5 scenarios, input, expected output.
 - **Agent Trace:** link to docs/sample_trace.json. Screenshot of trace screen.
 - **Assumptions:** bullet list from ASSUMPTIONS.md.
 - **Team:** table with name and role.
 - **Demo Video:** embed/link.
2. Write /docs/API_REFERENCE.md: document all 16 endpoints with example request and response bodies.
 3. Confirm all links in README work (no broken links).
 4. Commit: "docs: final README complete, API reference done"
 5.  **Deliverable:** Complete, professional README renders perfectly on GitHub.

Saad – Demo Dry Run & APK Build (09:00–13:00)

1. Run through the full demo script 3 times solo. Time each section.
2. Reset system state: call POST /simulate/reset. Verify clean slate.
3. Ensure phone is connected to same WiFi as backend. Update api_service.dart base URL to use your machine's local IP (e.g. http://192.168.1.X:8000) not 10.0.2.2.
4. Test Urdu signal input on physical device.
5. Build release APK: flutter build apk -release. Find APK at build/app/outputs/flutter-apk/app-release.apk.
6. Install APK on a fresh device (not the dev device). Run demo scenario. Confirm no crashes.
7. Take screenshots of key moments: crisis card, map before, map after, dashboard outcome, trace screen.
8. Save screenshots to /docs/DEMO_SCREENSHOTS/ with descriptive names.
9.  **Deliverable:** APK built, dry run complete, screenshots captured.

Shayan – Demo Video Recording (14:00–18:00, All Members)

1. **14:00 – All members join Google Meet.** Roles: Anas narrates, Saad runs mobile, Hasnain manages web dashboard, Arshman manages backend terminal (shows logs), Shayan operates OBS.
2. **Recording setup:** OBS at 1920x1080, 30fps, H.264, 6000kbps. Microphone: USB mic or headset (no laptop mic – too much noise).
3. **Record 3 takes.** No rushing. If a step goes wrong, reset and continue cleanly.
4. **Shayan edits the best take** (or combines two takes):
 - Add intro title card (5 seconds): “CIRO – Crisis Intelligence & Response Orchestrator | Google Antigravity Hackathon”
 - Add agent name text overlays when each agent activates
 - Add “BEFORE” / “AFTER” text overlays when simulation runs
 - Add outro title card (5 seconds): team names, GitHub link

- Cut dead air, keep total duration 3–5 minutes
- 5. Use DaVinci Resolve (free) or iMovie for editing. Export as MP4 H.264 1080p.
- 6. Upload to YouTube (Unlisted) OR Google Drive (shareable link). Verify link works incognito.
- 7. Add demo video link to README.md: " Demo Video: [Watch here](...)".
- 8. Final commit: "chore: demo video uploaded, link in README, v1.0 release ready"
- 9.  **End of Day Deliverable:** Demo video recorded, edited, exported, uploaded, linked.

📅 Day 6 – Tuesday, May 20 · Final Team Meet & Submission

Goal: All deliverables reviewed, packaged, and submitted. Every member has verified their component.

4.6 Day 6 – Final Meet & Submission

🕒 Schedule

10:00–10:15	Opening	Quick stand-up. Each member: “My component is ready because...”
10:15–11:00	Live Run	Run the full demo script together once – treat like it’s the real demo
11:00–11:30	Final Checks	Each member runs their individual checklist (below)
11:30–12:00	SUBMIT	Anas leads the submission upload process
12:00+	Celebration	Pack up and celebrate!

4.6.1 Individual Final Checklists

Anas: Inject test signal into Antigravity → verify all 5 agents fire in sequence with zero errors. Check Antigravity export YAML is in `/docs/`.

Hasnain: Open web dashboard in Chrome incognito (no cache). Click “Trigger Pipeline”. Verify all 4 panels update correctly within 15 seconds.

Arshman: Open README on GitHub. Verify every section renders, all images load, all links work. Check API reference is complete.

Saad: Install the release APK on a fresh device. Run Scenario 1 (Urdu flood). Verify map updates, dashboard updates, no crashes.

Shayan: Watch demo video end-to-end. Confirm duration is 3–5 minutes. Confirm audio is clear. Confirm link works for anyone.

4.6.2 Final Submission Checklist

- ✓ **Working Mobile App (MUST):** Flutter APK built and tested on physical device.
- ✓ **Working Web App (Optional):** Web dashboard accessible at `http://server:8000/web`.
- ✓ **Demo Video (3–5 min):** Uploaded to YouTube/Drive. Link in README. Shows all required content.
- ✓ **Agent Trace / Logs:** `/docs/sample_trace.json` committed. Screenshots in README.
- ✓ **Documentation (README):** Architecture, Antigravity usage, API docs, setup guide, assumptions, team.
- ✓ **GitHub Repository:** Clean commit history. All 5 members have commits. Repo is public or accessible.
- ✓ **Antigravity Config:** `/docs/antigravity-config.yaml` exported and committed.
- ✓ **API Reference:** `/docs/API_REFERENCE.md` covers all 16 endpoints.

5 Google Antigravity – Deep Integration Guide

⚠ Critical – 25% of Score

Antigravity accounts for **25% of the evaluation score**. This section ensures we maximise that 25%. Every team member must understand and be able to explain how we use Antigravity during the demo.

5.1 What We Use Antigravity For

1. **Multi-Agent Orchestration:** Antigravity defines the execution graph – which agent runs, in what order, what data it receives, and when to trigger the next agent. This replaces manual API chaining.
2. **Tool Integration:** Each agent has HTTP tools registered in Antigravity. When an agent needs to call a weather API or traffic endpoint, Antigravity handles the tool-calling protocol automatically.
3. **LLM Reasoning:** The Reasoning Agent uses **Gemini 1.5 Pro** via Antigravity. We pass the crisis event JSON as a prompt, and Gemini returns a structured JSON analysis.
4. **State Passing:** Antigravity passes Pydantic JSON schemas between agents automatically, eliminating manual handoff code.
5. **Built-in Trace:** Antigravity generates execution traces showing every agent step, tool call, and decision – which we surface in our Trace screen.

5.2 Agent Tool Registration Table

Agent	Tools Registered	Trigger Condition
signal-ingestion	POST /ingest/signal, GET /mock/social, GET /mock/weather, GET /mock/traffic	On new signal input
event-detection	POST /detect/crisis	On SignalBatch ready
reasoning-analysis	POST /reason/analyse, Gemini 1.5 Pro via Antigravity	On CrisisEvent confirmed
action-planning	POST /plan/actions	On CrisisAnalysis complete
simulation	POST /simulate/execute, GET /maps/crisis-overlay	On ActionPlan generated

5.3 How to Demo Antigravity to Judges

1. Open the Antigravity portal in a browser tab. Navigate to the **Live Workflow View**.
2. Trigger a signal from the web dashboard (“Trigger Pipeline” button).
3. Show the Antigravity canvas: each agent node **lights up in sequence** as it executes.
4. Switch to Antigravity’s **Trace tab**: show the full execution log with tool calls.

5. Narrate: *“This is Google Antigravity orchestrating our 5 AI agents in real-time. Each agent passes structured data to the next autonomously. The reasoning agent uses Gemini 1.5 Pro to analyse the crisis situation. No human intervention required.”*

6 Risk Management

Risk	Likelihood	Mitigation
Antigravity portal down	Medium	All agent logic exists in FastAPI. Pipeline can run as direct HTTP chain. Prepare this fallback demo mode.
Google Maps quota exceeded	Medium	All GeoJSON is pre-saved. Zero live API calls needed for demo. Switch to pre-saved files instantly.
Flutter crash on demo device	Low	Release APK tested on 2 devices. Screen recording backup of entire mobile demo ready.
Gemini API rate limit	Low	Gemini responses for all 5 scenarios pre-cached in <code>/data/gemini_cache.json</code> .
Team member unavailable	Low	All code documented with inline comments. Any member can explain any component.

7 Key Success Metrics

#	Metric	Target	Owner
1	Antigravity agents configured and firing	5 / 5	Anas
2	Signal types handled (social, weather, traffic)	3 / 3	Anas
3	Crisis types detected (flood, heat, blockage, accident)	4 / 4	Hasnain
4	Confidence scoring on multi-source test scenarios	≥ 0.80	Hasnain
5	Action types simulated (reroute, dispatch, alert, resource)	4 / 4	Arshman
6	Mobile app screens complete	5 / 5	Saad
7	Agent trace steps per run	5 / 5	Shayan
8	API endpoint response time	$< 500\text{ms}$	Hasnain
9	Demo video duration	3–5 min	Shayan
10	E2E test scenarios passing	5 / 5	Saad

8 All API Endpoints – Quick Reference

Method	Endpoint	Purpose
POST	/ingest/signal	Submit raw signal (text/JSON); returns SignalBatch
POST	/detect/crisis	Detect crisis from signal batch; returns CrisisEvent
POST	/reason/analyse	AI analysis via Gemini; returns CrisisAnalysis
POST	/plan/actions	Generate action plan; returns ActionPlan
POST	/simulate/execute	Execute simulation; returns SimulationResult
POST	/simulate/reset	Reset all mock state to defaults
GET	/simulate/state	Current mock system state (congestion, tickets, alerts)
GET	/simulate/tickets	List all emergency tickets
GET	/simulate/alerts	List all sent alerts
PATCH	/simulate/tickets/{id}/status	Update ticket status
GET	/maps/crisis-overlay	GeoJSON: crisis pin + affected area + routes
GET	/outcome/summary	Before/after outcome metrics
GET	/trace/latest	Full agent trace for most recent run
GET	/trace/history	Last 10 run summaries
GET	/mock/social	Random mock social media signal
GET	/mock/weather	Mock weather alert JSON
GET	/mock/traffic	Mock traffic congestion data
GET	/health	System health check

Let's Win This.

Five days. Five engineers. One mission.
Pakistan's cities need smarter crisis response.
We're building it.

    
Anas Hasnain Arshman Saad Shayan

CIRO – Crisis Intelligence & Response Orchestrator
Challenge 3 · Google Antigravity Hackathon · May 2025